UNIVERSITATEA TEHNICĂ
DIN CLUJ-NAPOCA

FACULTATEA: Automatică si Calculatoare
SPECIALIZAREA: Calculatoare si Tehnologia Informației
DISCIPLINA: Tehnici de programare
PROIECT: Calculator de polinoame
ASSIGNMENT NO: 1

Îndrumător laborator: Realizator:

Moldovan Dorin-Vasile Mihalache Rareș

Cuprins

1. Obiectivul temei 3
2. Analiza problemei, modelare, scenarii, cazuri de utilizare 3
3. Proiectare..... 4
   3.1. Proiectarea claselor 4
   3.2. Diagrama UML 5
4. Implementare..... 5
   4.1. Clase – Câmpuri și Metode 5
   4.2. Implementarea interfeței utilizator 8
5. Rezultate..... 10
   5.1. Testare 10
6. Concluzii 10
7. Bibliografie 10

1. Obiectivul temei

Obiectivul principal al acestei teme este de a realiza un calculator de polinoame. Acesta trebuie să fie dezvoltat pentru a facilita posibilitatea efectuării mai multor operații: adunare, scădere, înmulțire, împărțire, derivare, integrare.

Obiectivele secundare sunt:

- Extragerea polinoamelor din GUI cu ajutorul expresiilor Regex – Cap. 2
- Proiectarea claselor necesare pentru polinoame și monoame (core-ul aplicației) și a claselor necesare pentru arhitectura aplicației (MVC – Model, View, Controller) – Cap. 4
- Realizarea unei interfețe grafice pentru interacțiunea cu utilizatorul (GUI) – Cap. 4
- Modelarea logicii fiecărei operații – Cap. 4
- Testarea fiecărei operații pe exemple definite prin JUnit – Cap. 5

2. Analiza problemei, modelare, scenarii, cazuri de utilizare

Când utilizatorul pornește aplicația este intâmpinat de un Panel principal. Aici el poate să selecteze operația pe care dorește să o realizeze, apăsând pe un buton: “+” pentru adunare, “-” pentru scădere, “*” pentru înmulțire, “/” pentru împărțire, “ ’ ” pentru derivare și “∫” pentru integrare. După ce un buton a fost apăsat se reține într-o structură de date corespunzătoare (o listă de 6 elemente) această informație și se deschide o fereastră corespunzătoare pentru operația pe care utilizatorul dorește să o realizeze, înlocuind fereastra principală. Fiecare dintre aceste noi ferestre deschise are câte un buton de “Give result” care furnizează rezultatul dorit în funcție de operația aleasă și un buton de Back, pentru cazul în care utilizatorul vrea să meargă înapoi la meniul principal. (cel în care selectează operația dorită). Astfel, el nu este nevoit să redeschidă aplicația de fiecare dată când vrea să schimbe operația, ci poate naviga cu ușurință din operator în operator. Dacă butonul de Back a fost apăsat, structura de date (lista de operații) se resetează și nu se mai ține cont de butoanele care au fost apăsate anterior.

Fiecare fereastră nouă deschisă corespunde unui operator ales de utilizator. În funcție de acesta, fereastra are un design corespunzător. De exemplu, pentru operațiile de adunare, scădere, înmulțire și împărțire, utilizatorul este nevoit să introducă în 2 TextField-uri de input 2 polinoame. Rezultatul generat la apăsarea butonului de “Give result” se va afișa într-un TextField de output. Operatorii de derivare și integrare nu au decât un TextField de input și un TextField de output. Polinaomele introduse sunt sub forma unor String-uri și cu ajutorul expresiilor Regex putem să extragem informațiile individuale necesare și să separăm String-ul mare (polinomul) în subString-uri. Astfel, polinoamele vor fi separate în monoame, și apoi monoamele vor fi pe rând separate într-un coeficient real și un exponent întreg. Utilizatorul poate introduce polinoame în cadrul cărora se găsesc monoame de forma: “cx^p”, “c*x^p”, “x^p”, “cx”, “c*x”, “c”, “x”, unde:

- c -> coeficientul monomului, care este de tip int (poate avea semn negativ sau pozitiv)
- x -> variabila polinoamelor (poate fi orice caracter alfabetic – litere mici sau mari)
- p -> exponentul monomului, care este de tip int.

Între coeficient, variabilă și putere pot exista spații.

După ce se apasă pe butonul “Give result”, se pot întampla mai multe lucruri în funcție de care operație a fost aleasă:

- Utilizatorul a ales adunare => se descompun cele două String-uri din cadrul TextField-urilor de input în obiecte de tip Polynomial și se apelează funcția addPolynomials (Polynomial p1, Polynomial p2), iar rezultatul se transformă în String și se pune in TextField-ul de output.

- Utilizatorul a ales scădere => se descompun cele două String-uri din cadrul TextField-urilor de input în obiecte de tip Polynomial și se apelează funcția subtractPolynomials (Polynomial p1, Polynomial p2), iar rezultatul se transformă în String și se pune in TextField-ul de output.
- Utilizatorul a ales înmulțire => se descompun cele două String-uri din cadrul TextField-urilor de input în obiecte de tip Poynomial și se apelează funcția multiplyPolynomails (Polynomial p1, Polynomial p2), iar rezultatul se transformă în String și se pune în TextField-ul de output.
- Utilizatorul a ales împărțire => se descompun cele două String-uri din cadrul TextField-urilor de input în obiecte de tip Polynomial și se apelează funcția dividePolynomials (Polynomial p1, Polynomial p2), iar rezultatul se transformă în String. Aici există două componente: câtul și restul. Ele vor fi puse în TextField-ul de output sub forma "cât : rest".
- Utilizatorul a ales derivare => se descompune String-ul din cadrul TextField-ului de input în obiect de tip Polynomial și se apelează funcția derivatePolynomial (Polynomial p), iar rezultatul se transformă în String și se pune în TextField-ul de output.
- Utilizatorul a ales integrare => se descompune String-ul din cadrul TextField-ului de input în obiect de tip Polynomial și se apelează funcția integratePolynomial (Polynomial p), iar rezultatul se transformă în String și se pune în TextField-ul de output.

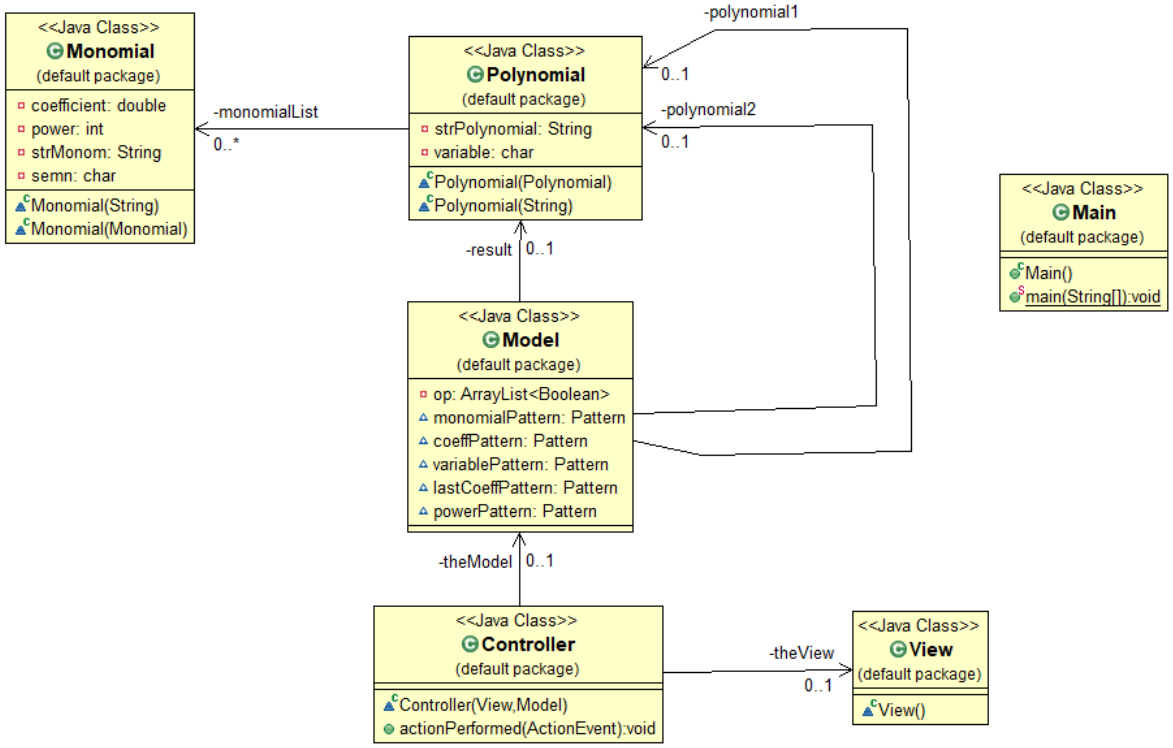
3. Proiectare

3.1. Proiectarea claselor

În cele ce urmează, vom descrie pe scurt fiecare clasă și ce responsabilități au acestea:

1. ***Clasa Main*** -> această clasă conține și funcția main și reprezintă locul de unde se începe execuția programului.
2. ***Clasa Monomial*** -> arată ideea de monom. Variabilele instanță semnificative sunt coeficientul (de tip double), puterea (de tip int), semnul(de tip char) și un câmp strMonom care reprezintă monomul sub formă de String. Această clasă are o varietate de metode care realizează operații pentru monoame și implementează interfața Comparable, utilă pentru sortarea în ordine descrescătoare după puteri a monoamelor.
3. ***Clasa Polynomial*** -> arată ideea de polinom. Variabilele instanță semnificative sunt formate din: o listă de monoame, un câmp care determină variabila polinomului (de tip char) si un câmp strPolynomial care reprezintă polinomul sub formă de String.
4. ***Clasa Model*** -> clasă care utilizează expresii Regex pentru a separa String-urile care reprezintă polinoame în monoame și mai departe in coeficienți, semne și puteri. În același timp, realizează conversia din String-urile din TextField-urile de input din GUI în obiecte de tip Polynomial. Aceste obiecte reprezintă operanzii folosiți pentru realizarea operației dorite. De asemenea, clasa Model este responsabilă pentru determinarea rezultatelor operațiilor și pentru transmiterea acestor rezultate la clasa Controller. Practic, această clasă reprezintă logica programului (alături de clasele Monomial și Polynomial).
5. ***Clasa View*** -> clasă care reprezintă interfața grafică utilizator. Extinde JFrame și implementează interfața ActionListener doar pentru a face tranziția dintre Panel-ul principal (meniul cu operatori) și celelalte două Panel-uri, atunci când este apăsat un buton (când este selectată o operație).
6. ***Clasa Controller*** -> mesager între clasa Model și clasa View. Implementează ActionListener pentru momentul când utilizatorul apasă pe butonul "Give result". În funcție de operația aleasă se furnizează rezultatul corect.

3.2. Diagrama UML



4. Implementare

4.1. Clase – Câmpuri și Metode

Clasa Monomial:

-<u>câmpuri:</u>

- coefficient (double)
- power (int)
- strMonom (String)
- semn (char)

-metode:

- listă mai mare de getters și setters:
  - ♦ public double getCoefficient();
  - ♦ public int getPower();
  - ♦ public String getStrMonom();
  - ♦ public char getSemn();
  - ♦ public void setCoefficient(double coefficient);
  - ♦ public void setPower(int power);
  - ♦ public void setSemn(char semn);

- public int compareTo (Monomial compareMon)
  - ➔ compară două monoame în funcție de exponentul lor

- public int addMonomial (Monomial m)
  - ➔ în cazul în care două monoame au puteri egale, le adună și returnează 1 (caz de succes), altfel returnează 0 (caz de esec)
- public int subtractMonomial (Monomial m)
  - ➔ în cazul în care două monoame au puteri egale, le scade si returnează 1 (caz de succes), altfel returnează 0 (caz de esec)
- public Monomial multiplyMonomial (Monomial m)
  - ➔ se face o instanță a clasei Monomial care reprezintă rezultatul. Se face înmulțirea dintre monomul curent (this) și monomul m și rezultatul se reține în noua instanță creată. Se returnează această instanță.
- public Monomial divideMonomial (Monomial m)
  - ➔ se face o instanță a clasei Monomial care reprezintă rezultatul. Se face împărțirea dintre monomul curent (this) și monomul m și rezultatul se reține în noua instanță creată. Se returnează această instanță.
- public void derivateMonomial()
  - ➔ se derivează monomul curent (this). Nu se returnează nimic.
- public void integrateMonomial()
  - ➔ se integrează monomul curent (this). Nu se returnează nimic.

Clasa Polynomial:

-câmpuri:

- ♦ monomialList (ArrayList<Monomial>)
- ♦ strPolynomial (String)
- ♦ variable (char)

-metode:

- listă mai mare de getters si setters:
  - ➔ public ArrayList<Monomial> getMonomialList();
  - ➔ public String getStrPolynomial();
  - ➔ public char getVariable();
  - ➔ public void setStrPolynomial(String strPolynomial);
  - ➔ public void setVariable(char variable);
- public void updatePolynomial()
  - ➔ se scot monoamele care au coeficientul egal cu 0 din lista de monoame a polinomului.
- public void transformToString()
  - ➔ se construiește câmpul strPolynomial al polinomului în funcție de lista de monoame a acestuia. (pentru operațiile de adunare, scădere, înmulțire și derivare (coeficienți întregi)).

- public void transformIntegrateToString()
  - ➔ se construiește câmpul strPolynomial al polinomului în funcție de lista de monoame a acestuia. (pentru operațiile de împărțire și integrare (coeficienți reali)).
- public void sortDescendingPowerPolynomial()
  - ➔ se ordonează lista de monoame dintr-un polinom în ordine descrescătoare dupa puterile acestora.
- public Polynomial addPolynomial (Polynomial p)
  - ➔ operație de adunare a polinoamelor "this" și "p". Returnează rezultatul.
- public Polynomial subtractPolynomial (Polynomial p)
  - ➔ operație de scădere a polinoamelor "this" și "p". Returnează rezultatul.
- public Polynomial multiplyPolynomial (Polynomial p)
  - ➔ operație de înmulțire a polinoamelor "this" și "p". Returnează rezultatul.
- public ArrayList<Polynomial> dividePolynomial (Polynomial dividend, Polynomial divisor)
  - ➔ operație de împărțire între două polinoame. Returnează un ArrayList<Polynomial> format din două polinoame: câtul și restul;
- public Polynomial derivate()
  - ➔ operație de derivare a polinomului "this". Returnează rezultatul.
- public Polynomial integrate()
  - ➔ operație de integrare a polinomului "this". Returnează rezultatul.

Clasa Model:

-<u>câmpuri:</u>

- ♦ Polynomial polynomial1
- ♦ Polynomial polynomial2
- ♦ ArrayList<Boolean> op

-<u>metode:</u>

- listă mai mare de getters și setters:
  - ➔ public Polynomial getPolynomial1()
  - ➔ public Polynomial getPolynomial2()
  - ➔ public ArrayList<Boolean> getOperationList()
  - ➔ public void setOperationList(ArrayList<Boolean> op)
  - ➔ public void setPolynomial1(String strPolynomial)
  - ➔ public void setPolynomial2(String strPolynomial)
- public String addPolynomials (Polynomial p1, Polynomial p2)
  - ➔ se folosește de metoda addPolynomial din clasa Polynomial pentru a aduna polinoamele p1 și p2 și returnează rezultatul sub formă de String clasei Controller.
- public String subtractPolynomials (Polynomial p1, Polynomial p2)
  - ➔ se folosește de metoda subtractPolynomial din clasa Polynomial pentru a face scăderea între polinoamele p1și p2 și returnează rezultatul sub formă de String clasei Controller.
- public String multiplyPolynomials (Polynomial p1, Polynomial p2)
  - ➔ se folosește de metoda multiplyPolynomial din clasa Polynomial pentru a înmulți polinoamele p1 și p2 și returnează rezultatul sub formă de String clasei Controller.
- public String dividePolynomials (Polynomial p1, Polynomial p2)
  - ➔ se folosește de metoda dividePolynomial din clasa Polynomial pentru a împărți polinoamele p1 și p2 și returnează rezultatul sub formă de String clasei Controller.
- public String derivatePolynomial (Polynomial p)
  - ➔ se folosește de metoda derivate din clasa Polynomial pentru a deriva polinomul p și returnează rezultatul sub formă de String clasei Controller.
- public String integratePolynomial (Polynomial p)
  - ➔ se folosește de metoda integrate din clasa Polynomial pentru a integra polinomul p și returnează rezultatul sub formă de String clasei Controller.

Clasa View:

Aici există 3 Panel-uri și mai multe componente de interfață grafică care se pun pe acestea. Metodele sunt în principal doar niște getters și o metodă actionPerformed care detectează ce operator a fost apăsat din Panel-ul principal și în funcție de acesta face tranziția spre un alt Panel. Butoanele de Back din celelalte două Panel-uri fac tranziția spre Panel-ul principal.

Clasa Controller:

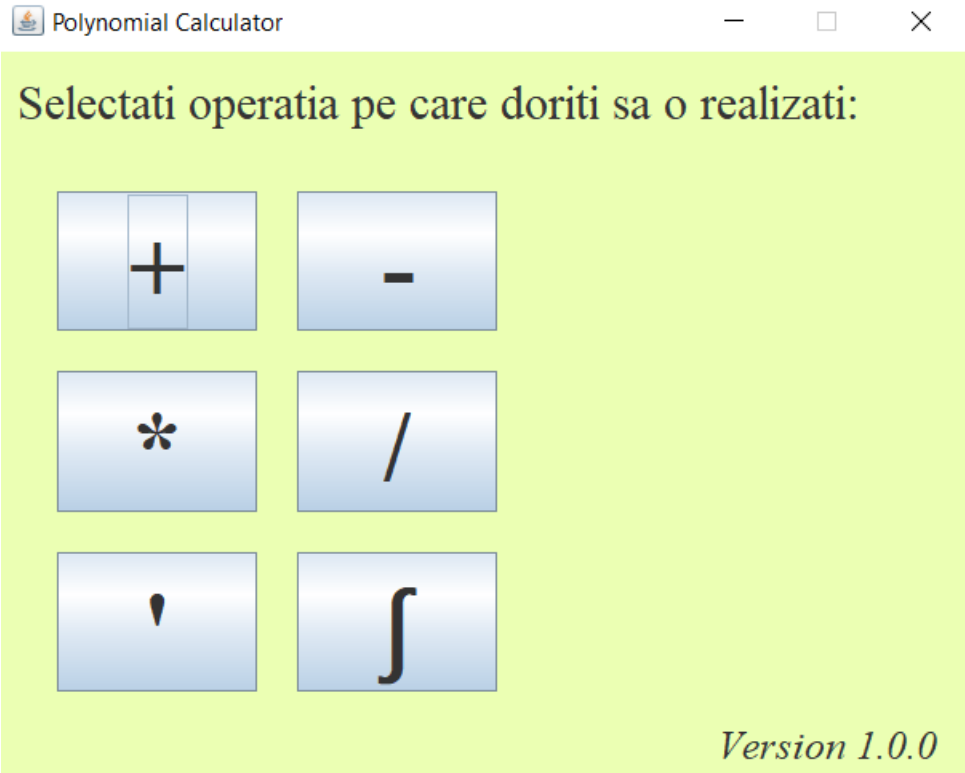
-<u>câmpuri:</u>

- View theView;
- Model theModel;

-<u>metode:</u>

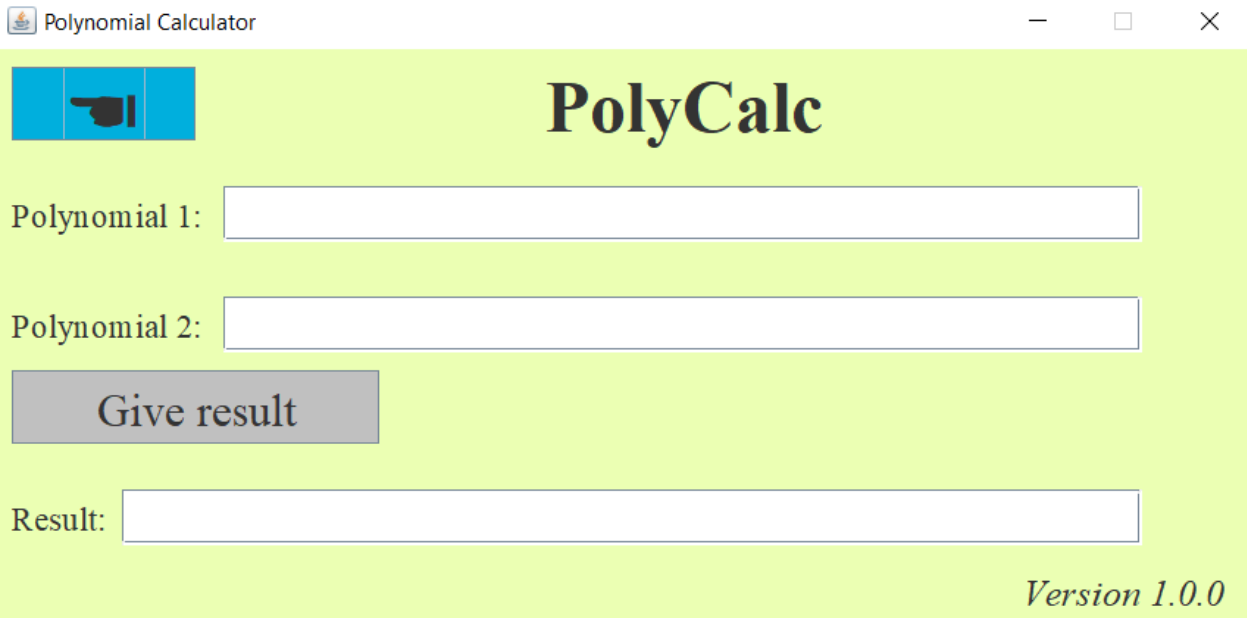
- public void actionPerformed(ActionEvent pressed)
  - ➔ face legătura dintre clasa Controller și clasa View și in functie de acel ArrayList<Boolean> din clasa Model selectează ce operație face și transmite rezultatul mai departe sub formă de String clasei View.

4.2. Implementarea interfeței utilizator



Interfața utilizator este User-Friendly. Când deschidem aplicația apare un Panel care conține mai multe butoane. Aici utilizatorul poate selecta operația pe care dorește să o realizeze. În funcție de câți operanzi are operația selectată, el va fi redirecționat către un alt Panel corespunzător.

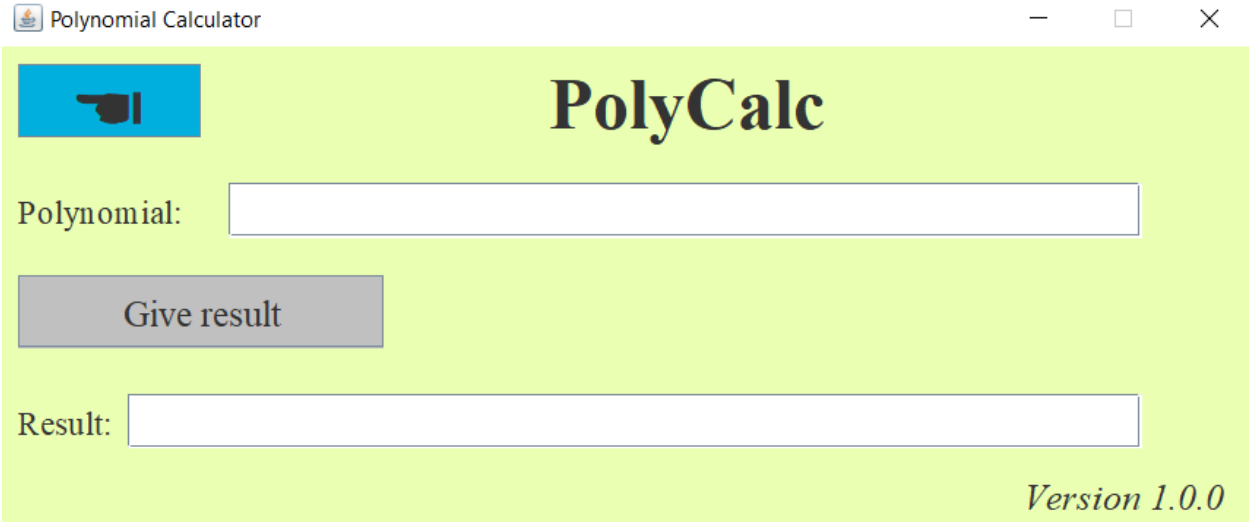
Presupunem că utilizatorul a ales una din operațiile: “+”, “-”, “*” sau “/”. Se deschide următorul Panel:



Dacă se apasă pe butonul de Back (cel cu background-ul albastru), ajungem la Panel-ul de început (cel cu operatori).

Aici, utilizatorul adaugă cele două polinoame în TextField-urile precedate de textul “Polynomial 1:” și “Polynomial 2:” și apasă pe butonul “Give result”. Polinomul rezultat este calculat și se afișează în TextField-ul precedat de textul “Result:”

Presupunem că utilizatorul a ales una din operațiile: “ ’ ” sau “∫”. Se deschide următorul Panel:



Dacă se apasă pe butonul de Back (cel cu background-ul albastru), ajungem la panel-ul de început (cel cu operatori).

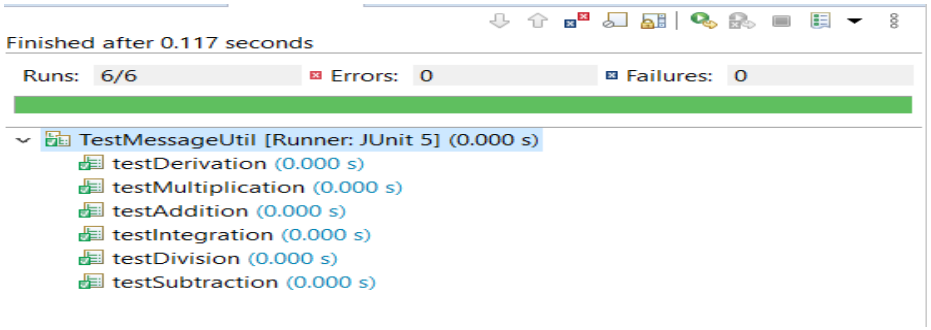
Aici, utilizatorul adaugă un polinom în TextField-ul precedat de textul “Polynomial:” și apasă pe butonul “Give result”. Polinomul rezultat este calculat și se afișează în TextField-ul precedat de textul “Result:”

5. Rezultate

5.1. Testare

Pentru a testa dacă rezultatele furnizate de operații sunt corecte am folosit o clasă de testare unitară: TestMessageUtil. Această clasă are o variabilă instantă de tip Model și mai multe metode pentru testarea fiecărei operații. Aceste metode sunt precedate de adnotarea @Test.

După ce se execută testele:



6. Concluzii

Această temă de laborator m-a ajutat să îmi reîmprospătez unele noțiuni dobândite semestrul trecut la POO și să văd concret aplicabilitatea acelor noțiuni în dezvoltarea unei aplicații. Această aplicație poate fi folosită pentru a calcula operații pe diferite polinoame. Au fost tratate mai multe cazuri pentru a testa dacă operațiile functionează bine și în marea majoritate a cazurilor merge cum trebuie. Am învățat și noțiuni noi cum ar fi expresii Regex și am văzut cum se lucrează cu ele. Proiectul m-a ajutat să consolidez concepte foarte importante din Java utlizate la dezvoltarea unei aplicații cum ar fi: arhitectura MVC, JUnit testing si Java Collections.

7. Bibliografie

1. https://regexr.com/
2. https://howtodoinjava.com/java/collections/arraylist/arraylist-sort-objects-by-field/
3. https://stackoverflow.com/questions/4066538/sort-an-arraylist-based-on-an-object-field
4. Laborator POO – sem. trecut
5. https://www.tutorialspoint.com/design_pattern/mvc_pattern.htm
6. https://www.w3schools.com/java/java_arraylist.asp